

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.serialRelease`

1.0.0-SNAPSHOT

September 2026



OSGi®

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
146	Serial Device Service Specification	7
146.1	Introduction	7
146.2	SerialDevice Service	8
146.3	SerialEventListener Service	9
146.4	USB Serial Example	9
146.5	Security	9
146.6	org.osgi.service.serial	9

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

146 Serial Device Service Specification

Version 1.0

146.1 Introduction

Recently, OSGi is gaining popularity as an enabling technology for building embedded systems in the residential market as well as other Internet-of-Things (IoT) domains. It is expected that communication with various devices attached to OSGi enabled gateways will be necessary.

Such communication can be implemented by means of serial connection when using non-IP devices based on ZigBee and Z-wave protocols. The most typical case arises when a USB dongle that supports such protocols is connected to the USB port of such a device, for example, residential gateway. The Operating System on the gateways will recognize the dongle as a virtual serial device and initiate a serial communication with the application process.

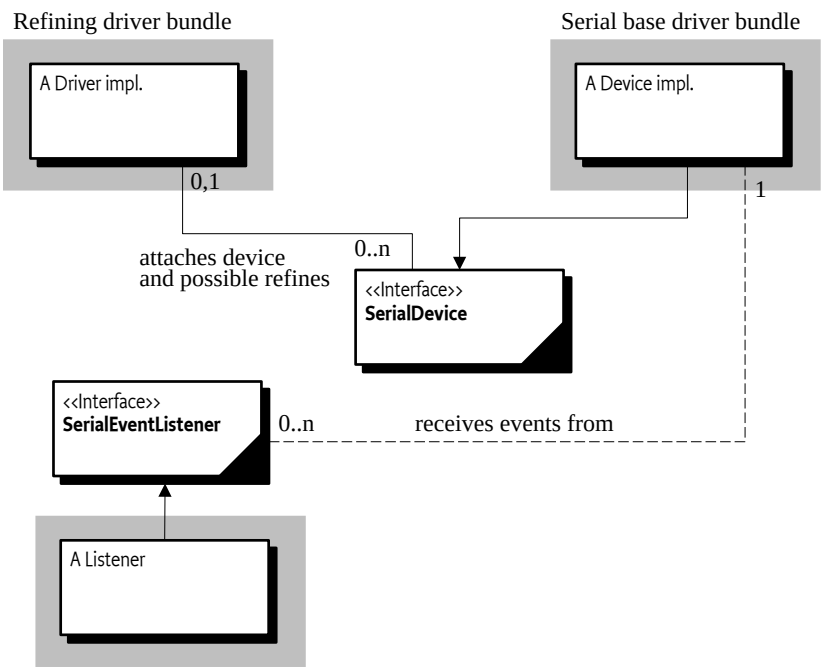
The Serial Device Service specification defines an API for establishing communications between an OSGi bundle and a serial device, such as a ZigBee coordinator or Z-Wave controller.

??? defines the concept of device categories. ??? defines a device category for USB devices. This specification and ??? provide a solution for the USB serial use case.

146.1.1 Entities

- *SerialDevice* - This is an OSGi service that is used to represent a serial device. This OSGi service stores information regarding serial device and its status as service properties and provides communication function with the device.
- *SerialEventListener* - A listener to events coming from Serial Devices.
- *Serial base driver bundle* - The bundle that implements [SerialDevice](#). Serial base driver bundle registers SerialDevice services with the Framework. It provides communication function with the (physical) serial devices.
- *Refining driver bundle* - Refining drivers provide a refined view of a physical device that is already represented by another Device service registered with the Framework (see the details for Device Access Specification).

Figure 146.1 Serial Device Service class diagram



146.2 SerialDevice Service

[SerialDevice](#) is the interface expressing a serial device. It maintains information and state of the serial device as a service property. It provides the communication facility with the serial device. Each [SerialDevice](#) expresses each serial device.

[SerialDevice](#) service is registered with the service registry with service properties as shown in the following table.

Table 146.1 Service properties of [SerialDevice](#) service

The key of service property	Type	Description
DEVICE_CATEGORY	String[]	Constant for the value of the service property <code>DEVICE_CATEGORY</code> used for all Serial devices. Value is "Serial".
serial.comport	String	MANDATORY property key. Represents the name of the port. Examples: <code>"/dev/ttyUSB0"</code> , <code>"COM5"</code> , <code>"/dev/tty.usbserial-XXXXXX"</code>

The Serial base driver may need native libraries. This document has a precondition that there are native libraries. It is out of scope how to install native libraries.

146.3 SerialEventListener Service

Serial events are sent using the white board model, in which a bundle interested in receiving the Serial events registers an object implementing the [SerialEventListener](#) interface. A COM port name can be set to limit the events for which a bundle is notified.

146.4 USB Serial Example

The Serial base driver registers a [SerialDevice](#) service that represents a (physical) Serial device. If the device is USB Serial device, then it is recommended that the base driver implements [??? and SerialDevice](#) concurrently, and registers the service under [??? and SerialDevice](#) interfaces.

146.5 Security

To acquire the Serial device service, the refining bundle need that `ServicePermission[SerialDevice, GET]` are assigned.

To receive the Serial events, the bundles need that `ServicePermission[SerialEventListener, REGISTER]` are assigned.

`SerialDevice` service should only be implemented by trusted bundles. This bundle requires `ServicePermission[SerialDevice, REGISTER]` and `ServicePermission[SerialEventListener, GET]`.

146.6 org.osgi.service.serial

Serial Device Service Specification Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.serial; version="[1.0,2.0)"}
```

146.6.1 Summary

- `SerialConstants` - Constants for serial device settings.
- `SerialDevice` - `SerialDevice` is a service representing a device performing serial communication.
- `SerialDeviceException` - A exception used to indicate that a serial device communication problem occurred.
- `SerialEvent` - A serial device event.
- `SerialEventListener` - Serial events are sent using the white board model, in which a bundle interested in receiving the Serial events registers an object implementing the `SerialEventListener` interface.
- `SerialPortConfiguration` - An object represents the Serial port configuration.

146.6.2 public final class SerialConstants

Constants for serial device settings.

- 146.6.2.1** **public static final int BAUD_115200 = 115200**
Baud rate: 115200.
- 146.6.2.2** **public static final int BAUD_14400 = 14400**
Baud rate: 14400.
- 146.6.2.3** **public static final int BAUD_19200 = 19200**
Baud rate: 19200.
- 146.6.2.4** **public static final int BAUD_38400 = 38400**
Baud rate: 38400.
- 146.6.2.5** **public static final int BAUD_57600 = 57600**
Baud rate: 57600.
- 146.6.2.6** **public static final int BAUD_9600 = 9600**
Baud rate: 9600.
- 146.6.2.7** **public static final int BAUD_AUTO = -1**
Baud rate: Automatic baud rate (if available).
- 146.6.2.8** **public static final int DATABITS_5 = 5**
Data bits: 5.
- 146.6.2.9** **public static final int DATABITS_6 = 6**
Data bits: 6.
- 146.6.2.10** **public static final int DATABITS_7 = 7**
Data bits: 7.
- 146.6.2.11** **public static final int DATABITS_8 = 8**
Data bits: 8.
- 146.6.2.12** **public static final int FLOWCONTROL_NONE = 0**
Flow control: None.
- 146.6.2.13** **public static final int FLOWCONTROL_RTSCS_IN = 1**
Flow control: RTS/CTS on input.
- 146.6.2.14** **public static final int FLOWCONTROL_RTSCS_OUT = 2**
Flow control: RTS/CTS on output.
- 146.6.2.15** **public static final int FLOWCONTROL_XONXOFF_IN = 4**
Flow control: XON/XOFF on input.
- 146.6.2.16** **public static final int FLOWCONTROL_XONXOFF_OUT = 8**
Flow control: XON/XOFF on output.
- 146.6.2.17** **public static final int PARITY_EVEN = 2**
Parity: Even.

146.6.2.18 **public static final int PARITY_MARK = 3**

Parity: Mark.

146.6.2.19 **public static final int PARITY_NONE = 0**

Parity: None.

146.6.2.20 **public static final int PARITY_ODD = 1**

Parity: Odd.

146.6.2.21 **public static final int PARITY_SPACE = 4**

Parity: Space.

146.6.2.22 **public static final int STOPBITS_1 = 1**

Stop bits: 1.

146.6.2.23 **public static final int STOPBITS_1_5 = 3**

Stop bits: 1.5.

146.6.2.24 **public static final int STOPBITS_2 = 2**

Stop bits: 2.

146.6.3 **public abstract SerialDevice**

SerialDevice is a service representing a device performing serial communication.

Concurrency Thread-safe

146.6.3.1 **public static final String DEVICE_CATEGORY = "Serial"**

Constant for the value of the service property {@code DEVICE_CATEGORY} used for all Serial devices.

A Serial base driver bundle must set this property key.

See Also org.osgi.service.device.Constants.DEVICE_CATEGORY

146.6.3.2 **public static final String SERIAL_COMPORT = "serial.comport"**

Service property for the serial comport.

Represents the name of the port. The value type is String.

For example, "/dev/ttyUSB0", "COM5", or "/dev/tty.usbserial-XXXXXX".

146.6.3.3 **public abstract SerialPortConfiguration getConfiguration()**

- Gets the Serial port configuration.

Returns The SerialPortConfiguration object containing the configuration.

146.6.3.4 **public abstract InputStream getInputStream() throws IOException**

- Returns an input stream.

Returns An input stream.

Throws IOException–if an I/O error occurred.

146.6.3.5 **public abstract OutputStream getOutputStream() throws IOException**

- Returns an output stream.

Returns An output stream.

Throws IOException – If an I/O error occurred.

146.6.3.6 public abstract boolean isCTS()

- Returns the CTS state.

Returns The CTS state.

146.6.3.7 public abstract boolean isDSR()

- Returns the DSR state.

Returns The DSR state.

146.6.3.8 public abstract boolean isDTR()

- Returns the DTR state.

Returns The DTR state.

146.6.3.9 public abstract boolean isRTS()

- Returns the RTS state.

Returns The RTS state.

146.6.3.10 public abstract void setConfiguration(SerialPortConfiguration configuration) throws SerialDeviceException

configuration The SerialPortConfiguration object containing the configuration.

- Sets the Serial port configuration.

Throws SerialDeviceException – If the parameter is specified incorrectly or the parameter is not supported.

146.6.3.11 public abstract void setDTR(boolean dtr) throws SerialDeviceException

dtr {@code true} for DTR on; {@code false} for DTR for off.

- Sets the DTR state.

Throws SerialDeviceException – If the parameter is not supported.

146.6.3.12 public abstract void setRTS(boolean rts) throws SerialDeviceException

rts {@code true} for RTS on; {@code false} for RTS for off.

- Sets the RTS state.

Throws SerialDeviceException – If the parameter is not supported.

**146.6.4 public class SerialDeviceException
 extends Exception**

A exception used to indicate that a serial device communication problem occurred.

146.6.4.1 public static final int PORT_IN_USE = 1

The port in use.

146.6.4.2 public static final int UNKNOWN = 0

The reason is unknown.

146.6.4.3 public static final int UNSUPPORTED_OPERATION = 2

The operation is unsupported.

146.6.4.4 public SerialDeviceException(int type, String message)

type The type for this exception.

message The message.

- Creates a SerialDeviceException with the specified type and message.

146.6.4.5 public int getType()

- Returns the type for this exception.

Returns The type of this exception.

146.6.5 public abstract SerialEvent

A serial device event. SerialEvent objects are delivered to SerialEventListeners when an event occurs.

A type of code is used to identify the event. Additional event types may be defined in the future.

Concurrency Thread-safe

146.6.5.1 public static final int DATA_AVAILABLE = 1

Event type indicating data available.

146.6.5.2 public abstract String getComPort()

- Returns the port name of this event.

This value must be equal to the value of {@link SerialDevice#SERIAL_COMPORT} service property of the {@link SerialDevice}.

Returns The port name of this event.

146.6.5.3 public abstract int getType()

- Returns the type of this event.

Returns The type of this event.

146.6.6 public abstract SerialEventListener

Serial events are sent using the white board model, in which a bundle interested in receiving the Serial events registers an object implementing the SerialEventListener interface. A COM port name can be set to limit the events for which a bundle is notified.

Concurrency Thread-safe

146.6.6.1 public static final String SERIAL_COMPORT = "serial.comport"

Key for a service property that is used to limit received events.

146.6.6.2 public abstract void notifyEvent(SerialEvent event)

event The SerialEvent object.

- Callback method that is invoked for received an event.

146.6.7 public class SerialPortConfiguration

An object represents the Serial port configuration.

Concurrency Immutable

146.6.7.1 public SerialPortConfiguration(int baudRate, int dataBits, int flowControl, int parity, int stopBits)

baudRate Baud rate.

dataBits Data bits.

flowControl Flow control.

parity Parity.

stopBits Stop bits.

- Creates an instance of the serial port configuration with the specified Baud rate, Data bits, Flow control, Parity and Stop bits.

146.6.7.2 **public SerialPortConfiguration(int baudRate)**

baudRate Baud rate.

- Creates an instance of the serial port configuration with the specified Baud rate and the following configuration: Data bits = 8, Flow control = none, Parity = none, Stop bits = 1.

146.6.7.3 **public SerialPortConfiguration()**

- Creates an instance of the serial port configuration with the following configuration: Baud rate = auto, Data bits = 8, Flow control = none, Parity = none, Stop bits = 1.

146.6.7.4 **public int getBaudRate()**

- Returns the baud rate.

Returns The baud rate.

146.6.7.5 **public int getDataBits()**

- Returns the data bits.

Returns The data bits.

146.6.7.6 **public int getFlowControl()**

- Returns the flow control.

Returns The flow control.

146.6.7.7 **public int getParity()**

- Returns the parity.

Returns The parity.

146.6.7.8 **public int getStopBits()**

- Returns the stop bits.

Returns The stop bits.

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT